opForge Reference Manual

This document describes the opForge assembler language, directives, and tooling. It follows a chapter layout similar to 64tass. Sections marked Planned describe features that are not implemented yet. This manual is validated against opForge CLI `0.9.7` (crate `0.9.7`).

1. Introduction

opForge is a two-pass, multi-CPU assembler. It currently ships builtin support for:

- Intel 8080 family processors: `8080` alias, `8085`, and `z80`
- MOS 6502 family processors: `6502`, `65c02`, `65816`, and `45gs02`
- Motorola 6800 family processors: `6809` and `hd6309`
- Motorola 68000 family processors: `68000`, `68010`, `68020`, `68030`, `68040`, and `68080`, with the corresponding `m68000`/`mc68000` through `m68080`/`mc68080` aliases

Optional FPU/coprocessor support is available for the Motorola 68000 family via the `.fpu` directive (`68881`, `68882`, `68040`, `68080`), and Apollo AMMX extensions are available on `68080` via `.apollo`.

It supports:

- Dot-prefixed directives and conditionals.
- A 64tass-inspired expression syntax (operators, precedence, ternary).
- Preprocessor directives for includes and conditional compilation.
- Macro expansion with `.macro` and `.segment`.
- Optional listing, Intel HEX, Motorola S-record, AmigaOS Hunk executable, and binary outputs.

The `.cpu` directive currently accepts:

- Intel 8080 family: `8080` (alias for `8085`), `8085`, `z80`
- MOS 6502 family: `6502`, `m6502`, `65c02`, `65816`, `65c816`, `w65c816`, `45gs02`, `m45gs02`, `mega65`, `4510`, `csg4510`
- Motorola 6800 family: `6809`, `m6809`, `mc6809`, `6309`, `m6309`, `h6309`, `hitachi6309`, `hd6309`
- Motorola 68000 family: `68000`, `m68000`, `mc68000`, `68010`, `m68010`, `mc68010`, `68020`, `m68020`, `mc68020`, `68030`, `m68030`, `mc68030`, `68040`, `m68040`, `mc68040`, `68080`, `m68080`, `mc68080`

1.1 Usage tips

- Directives, preprocessor directives, and conditionals are dot-prefixed (`.org`, `.if`, `.ifdef`).
- # prefixes immediate operands (for example `LDA #$10`).
- Macro invocation is dot-prefixed (for example `.COPY src,dst`).
- Labels may end with `:` or omit it.
- The program counter can be set with `* = expr` or `.org expr`.
- If no outputs are specified for a single input, the assembler defaults to list+hex when a root-module output name (or `-o`) is available.

2. Expressions and data types

2.1 Integers

Integer literals are supported in several formats:

- Decimal: `123`
- Hex: `$1234` or `1234h`
- Binary: `%1010` or `1010b`
- Octal: `17o` or `17q`

Underscores are allowed for readability (`$12_34`).

`$` evaluates to the current address.

2.2 Strings and escapes

Strings are quoted with `'` or `"` and are usable in data directives:

```
.byte "HELLO", 0
```

String bytes are encoded using the active text encoding (default `ascii`). Use `.encoding` or `.enc` to switch encodings, and `.encode`/`.cdef`/`.tdef`/`.edef` to define custom encodings in-source.

Strings accept the following escape sequences:

```
\n  \r  \t  \0  \xHH
```

Any other escape sequence inserts the escaped character as-is.

2.3 Booleans

Logical operators treat non-zero as true. Logical operators return `0` or `1`.

2.4 Symbols

Symbols are names bound to values. A symbol can be defined by a label (current address) or an assignment.

2.5 Expressions

Expressions are used in directives and operands. Unary < and > select the low or high byte of a value:

```
<($1234+1)
>($1234+1)
```

Safety rules:

- Division/modulo by zero reports a diagnostic error.
- Shift counts are bounded during evaluation.
- String literals in expression context are limited to 1-byte and 2-byte forms.
- Expression evaluation enforces a maximum recursion depth.

2.6 Expression operators

Operators listed from highest to lowest precedence:

Use this table when an expression mixes arithmetic, concatenation, shifts, and the ternary operator. When in doubt, add parentheses rather than relying on the reader to reconstruct precedence from memory.

Precedence	Operators	Description
1 (highest)	**	exponentiation (right-associative)
2	! ~ < >	unary: logical NOT, bitwise NOT, low byte, high byte
3	* / %	multiplication, division, modulo
4	+ −	addition, subtraction
5	..	concatenation
6	<< >>	bitwise shift left, shift right
7	< <= > >=	relational comparison
8	== != <>	equality, inequality
9	&	bitwise AND

Precedence	Operators	Description
10	^	bitwise XOR
11	|	bitwise OR
12	&& ^^	logical AND, logical XOR
13	||	logical OR
14 (lowest)	?:	ternary conditional

2.7 Conditional operator

The ternary operator `?:` is supported with standard precedence rules:

```
flag ? value_if_true : value_if_false
```

2.8 Compound values

opForge supports compile-time compound values in expressions:

These forms let source stay declarative when you want to build small tables, walk ranges, or carry typed field data through expressions rather than flattening everything into scalar constants.

- Ranges: `start..end`, `start..=end`, optional step `:step`.
- Lists: `{expr, expr, ...}`.
- Indexing: `value[n]` for range/list values.
- Member access: `value.field` when a struct type is associated with the value.
- Typed struct literals: `TypeName { field: expr, ... }`.
- Builtin length: `.len(expr)` for range/list values.

Dotted-name resolution order is:

1. exact dotted symbol lookup in the normal symbol namespace/scope;
2. typed member fallback (`base.field`) when no exact dotted symbol exists and `base` is struct-typed.

Examples:

```
vals = {1, 2, 3}
.byte vals[1]
.byte .len(0..=6:2)
```

Still planned (not implemented yet): bit strings, floating-point values, tuples, code blocks, and type values.

3. Compiler directives

3.1 Assembling source

```
.include "file"
.incbin "file"
```

Notes:

- `.include` is literal text inclusion only; it does not participate in module loading.
- `.incbin` includes the raw bytes of a binary file at the current program counter.
- Include resolution is constrained to the including-file directory and explicit `-I`/`--include-path` roots.
- Absolute include paths and parent-relative traversals are accepted only when they resolve inside those allowed roots.

3.2 Program counter and alignment

```
.org $1000
* = $2000
.align 16
```

- `.org` and `* =` set the program counter to the given address.
- `.align` pads the current emission target to a boundary.
- Both `.org` and `.align` apply to the current emission target (global or section-local).

3.3 Sections, regions, and placement

This is the part of the assembler that separates "where bytes are emitted" from "where bytes finally land". In small sources the distinction can be invisible, but once you have multiple output sections or Hunk memory classes it becomes the main layout tool.

A typical flow looks like this:

```
.region ram, $1000, $10ff, align=16
.section data, kind=data, align=2, region=ram
.section chipdata, kind=data, memory=chip
.endsection
.place data in ram
.pack in ram : code, data
```

The main rules are:

- `.section` selects a named emission target; `.endsection` restores the previous target.
- `.section` supports `kind=code|data|bss`, `align=<n>`, `region=<name>`, and `memory=any|chip|fast|slow`.
- `.place`/`.pack` assign final section base addresses via regions.
- `.dsection` is not supported and emits an error.
- Sections referenced by non-Hunk `.output` directives must be explicitly placed.
- `memory=` is used by AmigaOS Hunk output. `chip` and `fast` set the Hunk allocation flags for the emitted segment; `any` is the default. `slow` is accepted as a source-level alias for unconstrained allocation because the executable Hunk format has explicit CHIP and FAST bits, but no separate SLOW bit.
- For AmigaOS Hunk output, `.region`, `.place`/`.pack`, and `sections=` are optional for a simple single-code-hunk source. If no explicit `.section` is defined and `.output ..., format=hunk` omits `sections=`, opForge treats the flat emitted program bytes as one implicit `code` section. The same implicit code-hunk shorthand is used when Hunk output is requested with CLI `--hunk [FILE]` and the source has no explicit `.section`.
- `.mapfile` and `.exportsections` may include unplaced sections.

3.4 Output directives

Once sections have been emitted and placed, `.output` decides how that layout is serialized. Most projects only need one or two formats, but the same section graph can drive raw binaries, PRGs, S-records, or AmigaOS Hunks.

Common output declarations look like this:

```
.output "build/out.bin", format=bin, sections=code,data
.output "build/image.bin", format=bin, image="$8000..$80ff", fill=$ff, contiguous=false, sections
.output "build/out.prg", format=prg, sections=code
.output "build/out.srec", format=srec, sections=code
.output "build/tool.hunk", format=hunk
.mapfile "build/out.map", symbols=all
.exportsections dir="build/sections", format=bin, include=bss
```

Supported `format=` values:

- `bin` — raw binary image
- `prg` — Commodore PRG (2-byte little-endian load address prefix)
- `hunk` — AmigaOS executable Hunk format
- `srec` — Motorola S-record

The format chooses the container; the options around it control how strictly the placed sections must line up and what metadata is emitted alongside them.

Output mode rules:

- Default output mode is contiguous (`contiguous=true`): selected sections must be adjacent.
- Image output mode (`image=... + fill=...`) allows sparse placement within the configured span (wide addresses supported).
- PRG output prefixes a 2-byte little-endian load address (`loadaddr=` optional, must fit in 16 bits).
- Hunk output emits AmigaOS executable hunks. With explicit sections, `sections=` preserves the emitted Hunk segment order; without explicit sections, omitted `sections=` defaults to one code hunk from the flat source bytes. The shorthand also works in an implicit module; `.module` and `.endmodule` are not required just to declare the Hunk output. For no-frills CLI builds, `--cpu 68000 --hunk build/tool.hunk source.asm` can provide the target CPU and Hunk output path without requiring `.cpu` or `.output` in the source file. Hunk output also honors `.section ..., memory=chip|fast|slow|any` for selected sections.

The supported bare-symbol Hunk notation matrix is documented in §9 (Appendix: AmigaOS Hunk symbolic notation boundary).

3.5 Data directives

These directives cover the common "put bytes here" cases: scalar values, encoded text, repeated patterns, and reserved storage.

```
.byte expr[, expr...]
.db expr[, expr...]          ; alias for .byte
dc.b expr[, expr...]         ; 68000-style alias for .byte
.word expr[, expr...]
.dw expr[, expr...]          ; alias for .word
dc.w expr[, expr...]         ; 68000-style alias for .word
.long expr[, expr...]
dc.l expr[, expr...]         ; 68000-style alias for .long
.text "string"[, "string"...]
.null "string"
.ptext "string"
.ds expr
.emit unit, expr[, expr...]  ; unit = byte|word|long or numeric size
.res unit, count             ; BSS-only reservation
.fill unit, count, value     ; repeated data fill
```

The important constraints are:

- `.emit` and `.fill` are data-emitting directives and are not allowed in `kind=bss` sections.
- `.res` is only allowed in `kind=bss` sections.
- For `word`, unit size follows current CPU word size.
- String operands in `.byte`/`.db` are encoded using the active text encoding.
- `.null` is strict: it errors if the encoded source text already contains byte `0`.

3.5.1 Text encoding directives

```
.encoding name
.enc name                 ; alias for .encoding
.encode name[, base]
.endencode
.cdef start, end, value
.tdef chars, value[, value...]
.edef pattern, replacement[, replacement...]
```

Built-in encodings:

- `ascii` (default)
- `petscii`

Definition rules:

- `.encode` starts an encoding-definition scope and sets that encoding active.
- Optional `.encode name, base` clones `base` into `name` before applying new definitions.
- `.endencode` closes the scope and restores the previously active encoding.
- `.cdef` maps an inclusive source-byte range to consecutive output bytes.
- `.tdef` maps bytes from a source string:
  - two-operand form (`.tdef "abc", 32`) maps sequentially from the start value
  - multi-value form (`.tdef "abc", 1, 2, 3`) maps explicitly per character
- `.edef` defines escape substitutions evaluated before per-byte character mapping. Longest escape match wins.
- `.cdef`/`.tdef`/`.edef` are only valid inside `.encode ... .endencode`.

Compatibility notes:

- opForge follows 64tass-style encoding-definition directives, but keeps `.byte` string operands encoding-aware (no split behavior between `.byte` and `.text`).
- `.null` is strict: if encoded input already contains `0`, assembly fails.

3.6 Symbols and assignments

```
WIDTH = 40          ; read-only constant
var1  := 1          ; read/write variable
var2  :?= 5         ; only if undefined
var1  += 1          ; compound assignment
```

Compound assignment operators:

```
+= -= *= /= %= **= |= ^= &= ||= &&= <<= >>= ..= <?= >?= x= .=
```

`.const` and `.var` mirror = and := semantics; `.set` is an alias for `.var`.

For =, :=, :?=, `.const`, `.var`, and `.set`, symbol values may be scalar, list, or struct-instance values. Compound assignment operators (+=, −=, etc.) are scalar-only and reject non-scalar symbols.

3.7 Conditional assembly

```
.if expr
.elseif expr
.else
.endif
```

3.7.1 Match expressions

```
.match expr
.case expr[, expr...]
```

```
    ; body
.case expr
    ; body
.default
    ; body
.endmatch
```

The match expression is evaluated once; the first matching `.case` wins, and `.default` is used if no case matches.

3.7.2 Structured repetition opForge supports structured loop directives in assembler passes:

```
.for 4
    .byte $ff
.endfor

.for n in {1, 3, 5}
    .byte n
.endfor

.while expr
    ; body
.endwhile
```

Scoped repetition variants create per-iteration repeat scopes:

```
items .bfor i in 0..=2
value .byte i
.endfor

.bwhile expr
    ; body
.endwhile
```

Notes:

- Labels inside unscoped `.for`/`.while` bodies are rejected (use `.bfor`/`.bwhile`).
- Labels on `.endfor`/`.endwhile` are rejected.
- Loop iteration counts must be stable between pass1 and pass2.
- Loop execution is guarded by `--max-loop-iterations` (default `65536`, env override `OPFORGE_MAX_LOOP_ITERATIONS`).

3.8 Struct definitions

Struct definitions declare field offsets and total size:

```
Point .struct
x .byte ?
y .byte ?
.endstruct
```

Generated symbols:

- `Point`: struct size in bytes.
- `Point.x`, `Point.y`: field offsets.

When a labeled `.bfor` has inferred/annotated struct layout, indexed member access is available:

```
points .bfor i in 0..=1
x .byte i
y .byte i + 10
.endfor

.word points[1].x
.word points[1].y
```

Typed struct literal instances can be assigned to symbols:

```
Point .struct
x .byte ?
y .byte ?
.endstruct

p0 .const Point { x: 24, y: 50 }
p1 .var   Point { x: 40, y: 60 }
.byte p0.x, p1.y
p1 .set Point { x: 41, y: 61 }
```

Struct literal validation rules:

- struct type must exist;
- fields must be declared on the struct type;
- all fields must appear exactly once (no missing or duplicate fields).

Member resolution keeps a unified namespace:

- exact dotted symbols are resolved first;
- typed member access fallback is applied only when no exact dotted symbol exists.

Reference example: examples/struct_literal_instance_basic.asm

3.9 Scopes

Scopes provide symbol namespacing and are introduced by `.block` or `.namespace`. Both create hierarchical namespaces where symbols are qualified by their enclosing scope names, but they differ in naming requirements.

`.block` — Named or anonymous scopes `.block` creates a scope that can be either named (via a label) or anonymous:

```
; Named block (label becomes the scope name)
OUTER .block
VAL   .const 5
.endblock

; Anonymous block (internal name like __scope1)
      .block
LOCAL .const 3
      .endblock
```

Closed with `.endblock` or `.bend` (alias).

Use `.block` when you want local symbols that don't pollute the outer namespace, or when you need a named container for organizing related symbols.

`.namespace` — Always-named scopes `.namespace` creates a named scope and always requires a name, specified either as an operand or via a label:

```
; Name as operand
        .namespace outer
        .namespace inner
VAL     .const 9
        .endn           ; or .endnamespace
        .endnamespace

; Name via label
utils   .namespace
helper  .const 1
        .endn
```

Closed with `.endn` or `.endnamespace`.

Use `.namespace` when you specifically want to organize symbols into a named hierarchy, similar to namespaces in other languages.

Nesting and qualified access Both `.block` and `.namespace` can be nested and mixed:

```
OUTER .block
INNER .block
VAL   .const 5
.endblock
.endblock

        .word OUTER.INNER.VAL   ; qualified access from outside
```

Symbol resolution Symbol lookup searches in this order:

1. Current scope
2. Parent scopes (innermost to outermost)
3. Global scope

Inner scope symbols shadow outer symbols with the same name:

```
VAL    .const 1         ; global

SCOPE  .block
VAL    .const 2         ; shadows global VAL
       .word VAL        ; resolves to 2 (SCOPE.VAL)
       .endblock

       .word VAL        ; resolves to 1 (global)
       .word SCOPE.VAL  ; explicitly access inner (2)
```

Scope type matching Scope closers must match their openers:

- `.endblock`/`.bend` must close a `.block`
- `.endn`/`.endnamespace` must close a `.namespace`

Mismatched closers produce an error:

```
SCOPE .block
      .endnamespace     ; ERROR: opened by .block
```

Examples in the repo:

- examples/scopes.asm
- examples/scopes_namespace.asm

3.10 Modules and metadata

Modules define semantic scopes and imports; `.use` loads dependencies by module-id:

```
.module app.main
    .use util.math
    .pub
sum .const 0
.endmodule
```

Root-module metadata controls output naming:

```
.module main
    .meta
        .name "Demo Project"
        .version "1.0.0"
        .output
            .name "demo"
            .list
            .hex "demo-hex"
            .bin "0000:ffff"
            .fill "ff"
            .z80
                .name "demo-z80"
                .bin "0000:7fff"
                .fill "00"
            .endz80
        .endoutput
    .endmeta
    .meta.output.name "demo"
    .meta.output.z80.name "demo-z80"
.endmodule
```

Inside a `.meta` block, `.name` sets the metadata name. Inside an `.output` block (or `.meta.output.*` inline), `.name` sets the output base name. `.list`/`.hex`/`.bin`/`.fill` are valid only inside `.output` blocks or via `.meta.output.*` inline directives.

`.use` forms (module scope only):

```
.use util.math
.use util.math as M
.use util.math (add16, sub16 as sub)
.use util.math with (FEATURE=1, MODE="fast")
```

Notes:

- `.use` must appear inside a module and at module scope.
- `.use` affects runtime symbol resolution only.
- `.pub`/`.priv` visibility is enforced for runtime symbols (labels/constants/vars) only; macro/segment exports are not filtered by `.use`.

3.10.1 Root input

- `-i` accepts a file or folder.

- Folder input must contain exactly one `main.*` file (case-insensitive, `.asm` or `.inc`).
- The folder name becomes the input base (used for default output names).

3.10.2 Module identity

- If a file has no explicit `.module`, it defines an implicit module whose id is the file basename.
- If explicit modules exist, all top-level content must be inside `.module` blocks.
- The root module is:
  - the module matching the entry filename (case-insensitive), or
  - the first explicit module if no match exists.

3.10.3 Module resolution

- Search root: entry file directory only.
- Extensions: fixed to `.asm` and `.inc`.
- Module id matching is case-insensitive.
- If a file defines multiple modules, only the requested module is extracted.
- Missing or ambiguous module ids are errors; errors include an import stack.

3.10.4 Visibility rules

- `.pub`/`.priv` control runtime symbol visibility (labels/constants/vars).
- Macro/segment exports are not filtered by `.use`.

3.10.5 Root metadata output rules Output base precedence:

1. `-o/--outfile`
2. `.meta.output.<target>.name`
3. `.meta.output.name`
4. input base (file basename or folder name)

Examples in the repo:

- examples/module_use_autoload.asm
- examples/module_metadata_output.asm
- examples/project_root/main.asm

3.11 Target CPU

The `.cpu` directive selects the active instruction set, operand rules, and any family-specific assembler behavior. The accepted identifiers are easiest to read when grouped by CPU family.

3.11.1 Intel 8080 family

```
.cpu 8080    ; alias for 8085
.cpu 8085
.cpu z80
```

Use these selectors for Intel 8080/8085 syntax and the Z80 dialect surface.

3.11.2 MOS 6502 family

```
.cpu 6502
.cpu m6502
.cpu 65c02
.cpu 65816
```

```
.cpu 65c816
.cpu w65c816
.cpu 45gs02
.cpu m45gs02
.cpu mega65
.cpu 4510
.cpu csg4510
```

This family spans baseline 6502 syntax through 65C02, 65816, and Mega65/4510 extensions.

3.11.2.1 65816 runtime-state directives 65816 support includes the phase-1 instruction set and phase-2 24-bit addressing work:

- Implements selected 65816 mnemonics and operand forms.
- Includes long memory forms for `ORA`, `AND`, `EOR`, `ADC`, `STA`, `LDA`, `CMP`, and `SBC` (`$llhhhh` and `$llhhhh,X`).
- Includes stack-relative forms (`d,S` and `(d,S),Y`) for `ORA`, `AND`, `EOR`, `ADC`, `STA`, `LDA`, `CMP`, and `SBC`.
- Includes wide-address output/layout workflows (`.org`, `.region`, `.place`, `.output image=...`, HEX/BIN emission).
- Includes REP/SEP-driven M/X width-state tracking for supported width-sensitive immediate mnemonics.

`.assume` directive:

```
.assume e=0, m=8, x=8, dbr=$7e, pbr=$80, dp=$1000
.assume dbr=auto
.assume pbr=auto
```

`.assume` sets explicit 65816 runtime-state assumptions. The assembler uses these assumptions to resolve ambiguous addressing modes (for example absolute-vs-long and direct-page offset selection).

- Includes explicit per-operand mode overrides for ambiguous bank/page forms: `,d`, `,b`, `,k`, and `,l`.
- Uses current assembly address bank as the default PBR assumption for `JMP`/`JSR` absolute-bank resolution when `.assume pbr=...` is not set.

Mode-selection precedence:

For ambiguous bank/page-sensitive operands, opForge resolves in this order:

1. explicit operand override (`,d`, `,b`, `,k`, `,l`)
2. global `.assume` state (`dbr`, `pbr`, `dp`, plus `e`/`m`/`x` for widths)
3. automatic deterministic fallback

Conservative state invalidation:

- `PLB` invalidates known `DBR`.
- `PLD` and `TCD` invalidate known `DP`.
- `.assume dbr=auto` / `.assume pbr=auto` clear explicit bank overrides and return to inferred bank behavior.

Migration note:

- Source that previously relied on stack-sequence inference (`PHK`/`PLB`, `LDA #imm ... PHA ... PLB`, `PEA ... PLB`, and related `... PLD` patterns) should be updated to use explicit operand overrides and/or local `.assume` updates at the relevant call sites.

Current 65816 limits:

- PRG load-address prefix remains 16-bit.
- Full automatic banked-state inference is not implemented (`.assume` plus explicit overrides provide control).

- Uses checked address arithmetic and explicit diagnostics for overflow/underflow paths in placement, linking, and image emission.

3.11.3 Motorola 6800 family

```
.cpu 6809
.cpu m6809
.cpu mc6809
.cpu 6309
.cpu m6309
.cpu h6309
.cpu hitachi6309
.cpu hd6309
```

These selectors cover the Motorola 6809 lineage and the HD6309-compatible variants.

3.11.4 Motorola 68000 family

```
.cpu 68000
.cpu m68000
.cpu mc68000
.cpu 68010
.cpu m68010
.cpu mc68010
.cpu 68020
.cpu m68020
.cpu mc68020
.cpu 68030
.cpu m68030
.cpu mc68030
.cpu 68040
.cpu m68040
.cpu mc68040
.cpu 68080
.cpu m68080
.cpu mc68080
```

Motorola 68000-family support spans the shipped CPU lineage from `68000` through `68080`.

`68010` keeps baseline `68000` addressing. `68020`, `68030`, and `68040` accept the shipped `68020+` full-extension addressing forms. `68040` additionally accepts `MOVE16` and rejects `CALLM`, `RTM`, and `MOVEC CAAR`.

The shipped `68080` surface includes the full currently documented integer, AMMX, and legacy FPU assembler-visible families, including E/B register namespaces and `.fpu 68080`.

3.11.4.1 FPU and coprocessor selection

```
.fpu none
.fpu 68881
.fpu 68882
.fpu 68040
.fpu 68080

.apollo on
.apollo off
```

On Motorola 68000-family CPUs, `.fpu none` disables optional FPU acceptance, `.fpu 68881` and `.fpu 68882` are legal on `68020` and `68030`, and `.fpu 68040` is legal on `68040`, while `.fpu 68080` is legal on `68080`.

On `68080`, `.cpu 68080` defaults to the Apollo-enabled full profile; `.apollo on` is accepted as an explicit no-op, and `.apollo off` is rejected deterministically because strict compatibility mode is not implemented in this full-profile build.

Current MMU scope remains intentionally narrow: `PFLUSH` is accepted on `68030` and `68040`, and the shipped `68040` MMU-related `MOVEC` register surface remains available. Broader PMMU/MMU instruction families stay out of scope.

Current FPU scope is selector-driven and assembler-only. `.fpu 68881` and `.fpu 68882` enable the external coprocessor surface on `68020` and `68030`, while `.fpu 68040` enables the integrated `68040` core FPU subset on `68040`, and `.fpu 68080` is the integrated target on `68080`. The `68040` integrated path intentionally excludes external-coprocessor-only `FSIN`-class transcendental and extended-math mnemonics. Assembler acceptance follows the documented programmer-visible instruction surface, but opForge does not model runtime assist behavior or CPU/FPU execution semantics for those operations.

Reference fixtures under `examples/motorola68000/` include broad FPU surface examples such as `68020_fpu_allmodes`, `68020_fpu_instruction_catalog`, `68020_fpu_registers`, `68030_pflush_external_fpu`, and `68040_integrated_fpu`, alongside 68080-focused fixtures such as `68080_integer_addressing_matrix`, `68080_ammx_addressing_matrix`, `68080_fpu_surface`, and `68080_full_additional_surface`, with matching checked-in outputs under `examples/reference/motorola68000/`.

3.12 End of assembly

```
.end
```

3.13 Preprocessor directives

Preprocessor directives are dot-prefixed:

```
.ifdef NAME
.ifndef NAME
.elseif NAME
.else
.endif
.include "file"
```

Notes:

- # is used for immediate operands; preprocessor directives must use dot form.
- Preprocessor directives run before macro expansion.
- Preprocessor symbols are provided via the `–D/––define` command-line option.

4. Pseudo instructions

4.1 Macros

```
NAME .macro a, b=2
    .byte .a, .b
.endmacro
```

`.endm` is an alias for `.endmacro`.

Alternate directive-first form:

```
.macro NAME(a, b=2)
    .byte .a, .b
.endmacro
```

Invoke with `.NAME`:

Parenthesized call form:

```
.NAME(1)
```

4.2 Macro parameters

- Positional: `.1` .. `.9`
- Named: `.name` or `.{name}`
- Full argument list: `.@`
- Text form: `@1` .. `@9`

4.3 Segment macros

`.segment` defines a macro that expands inline without an implicit `.block`/`.endblock` wrapper:

```
INLINE .segment v
    .byte .v
.endsegment

.INLINE 7
```

Alternate directive-first form:

```
.segment INLINE(v)
    .byte .v
.endsegment
```

`.ends` is an alias for `.endsegment`.

4.4 Statement patterns (`.statement`)

`.statement` defines a patterned statement signature that is matched when the statement label appears without a leading dot:

```
.statement move.b char:dst "," char:src
    .byte 'b'
    .byte '.dst', 0
    .byte '.src', 0
.endstatement

move.b d0, d2
```

Rules:

- Typed captures use the explicit `type:name` form (e.g. `byte:val`, `char:reg`).
- Literal commas must be quoted as `","` inside signatures.
- Statement labels may include dots (e.g. `move.b`, `move.l`).
- Boundary spans `[{ ... }]` enforce adjacency rules within the span.

Capture types (built-in):

- `byte`, `word`, `long`, `char`, `str`
- `byte` matches a single token that is either a numeric literal in `−128..=255`, an identifier/register, or a 1-byte quoted string.

- `word` matches a single token that is either a numeric literal in −32768`..`=65535, an identifier/register, or a 1-2 byte quoted string.
- `long` matches a single token that is either a numeric literal in −2147483648`..`=4294967295 or an identifier/register. Quoted strings do not match `long`.
- Capture matching is single-token only. Multi-token expressions such as `label+4` or `1<<24` do not match `long` captures.

Reference example: examples/statement_expansion.asm

Expansion model:

- `.statement` definitions are expanded by the macro processor before parsing.
- Statement definitions are global (not module-scoped).

4.5 Scope and namespace interaction (**`.macro`**, **`.segment`**, **`.statement`**)

This section describes how compile-time definition lookup and expansion-time symbol scopes interact.

Definition-time behavior:

- `.macro` and `.segment` definitions are namespace-aware:
  - names are qualified by the active `.namespace` stack
  - lookup prefers the nearest namespace, then outer namespaces, then global
- `.block` does not add macro/segment name qualification.
- `.statement` definitions are global keyword entries (not namespace- or module-scoped).

Expansion-time behavior:

- `.macro` expands with an implicit `.block` / `.endblock` wrapper.
  - Symbols created inside the expanded body are scoped to that generated block.
  - If the invocation has a label, that label is attached to the generated `.block` opener.
- `.segment` expands inline with no implicit scope wrapper.
  - Labels/symbol assignments in the body resolve in the caller's current scope.
- `.statement` expansions are also inline (no implicit `.block`).
  - Use an explicit `.block` inside the statement body when local symbol isolation is required.

Definition constraints:

- Nested `.macro`/`.segment` definitions are not supported.
- Nested `.statement` definitions are not supported.
- `.statement` cannot be defined inside `.macro` or `.segment` bodies.

Example:

```
.namespace outer

ADD1 .macro x
tmp .const 1        ; local to generated implicit .block
    .byte .x + tmp
.endmacro

INLINE .segment x
tmp .const 2        ; resolves in caller scope (no implicit block)
    .byte .x + tmp
.endsegment

.statement emit1 byte:v
tmp .const 3        ; resolves in caller scope unless body adds explicit .block
    .byte .v + tmp
```

```
.endstatement

.endn
```

5. Command line options

Syntax:

```
opForge [OPTIONS] [INPUT]...
```

The CLI is easiest to read in four groups: how input is discovered, how output is produced, how diagnostics are rendered, and a smaller set of switches that adjust assembler behavior.

Inputs:

- `[INPUT]...`: optional migration-friendly positional input. Exactly one positional input is accepted and treated like `-i INPUT`; multiple positional inputs require explicit `-i/--infile`.
- `-i, --infile <FILE|FOLDER>`: input `.asm` file or folder (repeatable). Folder inputs must contain exactly one `main.*` root module.
- `-I, --include-path <DIR>`: additional include search roots (repeatable). Include resolution order: including-file directory, then include roots in CLI order.
- `-M, --module-path <DIR>`: additional module search roots (repeatable). Module resolution order: input root directory, then module roots in CLI order.

Outputs:

- `-l, --list [FILE]`: listing output (optional filename).
- `-x, --hex [FILE]`: Intel HEX output (optional filename).
- `-s, --srec [FILE]`: Motorola S-record output (optional filename).
- `--hunk [FILE]`: AmigaOS Hunk executable output (optional filename).
- `-b, --bin [FILE:ssss:eeee|ssss:eeee|FILE]`: binary image with optional range(s), repeatable (`ssss`/`eeee` are 4-8 hex digits).
- `--dependencies <FILE>`: write Makefile-compatible dependency rules.
- `--dependencies-append`: append dependency rules to an existing dependency file.
- `--make-phony`: emit phony targets for dependency paths.
- `--labels <FILE>`: write symbol labels.
- `--vice-labels`: write `--labels` output in VICE-compatible format.
- `--ctags-labels`: write `--labels` output in ctags-compatible format.

Diagnostics:

- `--format <text|json>`: select diagnostic output format (default: `text`).
- `--diagnostics-style <rustc|classic>`: select diagnostic rendering style (default: `rustc`).
- `-q, --quiet`: suppress diagnostics for successful assembly runs.
- `-E, --error <FILE>`: route diagnostics to file instead of stderr.
- `--error-append`: append diagnostics to `--error` file.
- `--no-error`: disable diagnostic routing.
- `-w, --no-warn`: suppress warning diagnostics.
- `--Wall`: enable all warning classes (reserved for future groups).
- `--Werror`: treat warnings as errors.

Other options:

- `-o, --outfile <BASE>`: output base name if output filename omitted.
- `-f, --fill <hh>`: fill byte for binary output (hex). Requires binary output. Defaults to `FF`.
- `-g, --go <aaaa>`: execution start address in HEX/S-record output (4-8 hex digits). Requires HEX or S-record output.
- `-D, --define <NAME[=VAL]>`: predefine macro (repeatable).
- `-c, --cond-debug`: include conditional state in listing.

- `--line-numbers`: listing compatibility flag for line-number column (enabled by default).
- `--tab-size <N>`: expand tab characters in listing source column.
- `--verbose-list`: listing compatibility flag (reserved for expanded listing sections).
- `--fmt`: format input files in place (shorthand for `--fmt-write`). Folder inputs also include linked module files.
- `--fmt-check`: check formatting for input files without writing changes. Folder inputs include linked module files.
- `--fmt-write`: apply formatter changes in place for input files. Folder inputs include linked module files.
- `--fmt-stdout`: format exactly one input file and write the result to stdout.
- `--fmt-config <FILE>`: formatter config file path (requires a formatter mode flag).
- `--cpu <ID>`: select initial CPU before source parsing (`.cpu` in source can still override later).
- `--opasm-package <FILE>`: load runtime `.opasm` package from FILE and prefer it over artifact/bundled package sources.
- `--print-capabilities`: print deterministic capability metadata and exit.
- `--print-cpusupport`: print deterministic CPU support metadata and exit.
- `--pp-macro-depth <N>`: maximum preprocessor macro expansion depth (default 64, minimum 1).
- `--max-loop-iterations <N>`: maximum `.for`/`.while` iterations before emitting an error (default 65536, minimum 1).
- `--input-asm-ext <EXT>`: additional accepted source-file extension for direct file inputs.
- `--input-inc-ext <EXT>`: additional accepted root-module extension for folder inputs.
- `-h, --help`: print help.
- `-V, --version`: print version.

The default capability and CPU-support reports include the shipped Motorola 68000-family lineage entries through `m68080`.

The following rules are the ones most likely to affect day-to-day invocation:

- If multiple inputs are provided, `-o` must be a directory and explicit output filenames are not allowed; each input uses its own base name under the output directory.
- With multiple inputs, at least one output type (`-l`, `-x`, `-s`, `--hunk`, `-b`) must be selected.
- If no outputs are specified for a single input, opForge defaults to list+hex when `.meta.output.name` (or `-o`) is available; otherwise output selection is required.
- Relative output filenames are anchored to the input file's directory.
- Formatter mode (`--fmt`, `--fmt-check`, `--fmt-write`, `--fmt-stdout`) requires at least one input and cannot be combined with assembler output flags or fixit options.
- `--fmt-stdout` requires exactly one input.
- `-b` without a range emits a binary that spans the emitted output.
- For Intel HEX, `-g` writes a Start Segment Address record for 16-bit values and a Start Linear Address record for wider values.
- For S-record output, `-g` writes the start address into the S7/S8/S9 termination record; without `-g`, the termination address is zero.
- `--hunk` does not currently consume `-g`; AmigaOS executable hunks remain loader-relocatable and use the Hunk segment/relocation records emitted by the Hunk writer.

5.1 Formatter configuration

The formatter operates in four modes: `--fmt` (in-place, shorthand for `--fmt-write`), `--fmt-write` (in-place), `--fmt-check` (dry-run), and `--fmt-stdout` (single file to stdout). Formatter mode cannot be combined with assembler output flags.

The current formatter surface is intentionally small and conservative. The goal is predictable cleanup of layout and casing, not large-scale source rewriting.

Formatter config (`--fmt-config`) currently supports these keys:

```toml
[formatter]
profile = "safe-preserve"            # only supported profile in Phase 1
preserve_line_endings = true
preserve_final_newline = true
label_alignment_column = 8           # alias: code_column
max_consecutive_blank_lines = 1      # alias: max_blank_lines
align_unlabeled_instructions = true  # align unlabeled opcodes to code column (data directives
split_long_label_instructions = true  # if label exceeds column, move mnemonic to next line
label_colon_style = "keep"           # keep|with|without
directive_case = "keep"              # keep|upper|lower
label_case = "keep"                  # keep|upper|lower
mnemonic_case = "keep"               # keep|upper|lower (alias: opcode_case)
register_case = "keep"               # keep|upper|lower
hex_literal_case = "keep"            # keep|upper|lower
```

Validation is strict:

- unknown keys are errors
- duplicate keys are errors (including alias duplicates)
- invalid value types are errors
- unsupported `profile` values are errors
- without `--fmt-config`, formatter runs always use built-in defaults and do not auto-discover `.op-forgefmt.toml`

V2 note: `label_case` is planned to become symbol-aware so label usage tokens are case-normalized alongside label definitions.

6. Messages

Diagnostics include a line/column and a highlighted span in listings. Terminal output may use ANSI colors to highlight the offending region.

Listing addresses reflect the current emission context: inside a `.section` the address column shows the section-local program counter. Absolute placed output is shown in the generated-output footer table.

In practice, the most common diagnostics in this area are placement mistakes:

- `.dsection has been removed; use .place/.pack with .output`
- `Section referenced by .output must be explicitly placed`
- `contiguous output requires adjacent sections` (gap diagnostic includes range)

7. Appendix: quick reference

This appendix is meant for scanning, not teaching. Use it when you already know the feature you want and just need the exact directive name or operator form.

7.1 Directives

```
.org  .align  .region  .place  .pack  .section  .endsection  .cpu  .end
.fpu  .apollo  .assume
.encoding  .enc  .encode  .endencode  .cdef  .tdef  .edef
.byte  .db  .word  .dw  .long  .text  .null  .ptext  .ds  .emit  .res  .fill
.const  .var  .set
.if  .elseif  .else  .endif  .match  .case  .default  .endmatch
.for  .bfor  .endfor  .while  .bwhile  .endwhile
.struct  .endstruct
```

```
.ifdef  .ifndef  .include  .incbin
.module .endmodule .use .pub .priv .block .endblock .bend .namespace .endn .endnamespace
.macro  .endmacro  .endm  .segment  .endsegment  .ends  .statement  .endstatement
.meta .endmeta .name .version .output .endoutput .list .hex .bin .mapfile .exportsections
.meta.name  .meta.version
.meta.output.name .meta.output.<target>.name .meta.output.list .meta.output.hex .meta.output.b
```

7.2 Assignment operators

```
=  :=  :?=  +=  -=  *=  /=  %=  **=  |=  ^=  &=  ||=  &&=  <<=  >>=  ..=  <?=  >?=  x=  .=
```

7.3 Scope behavior at a glance

The table below compresses the behavior described in Section 4.5 into a single comparison so you can quickly answer ”what kind of scope does this definition create?” without rereading the full discussion.

Construct	Definition lookup	Expansion form
`.macro`	Namespace-aware (nearest `.namespace` first, then outer, then global)	Implicit `.block` ... `.endblock` wra
`.segment`	Namespace-aware (same lookup as `.macro`)	Inline expansion (no implicit wrapp
`.statement`	Global keyword registry (not namespace-scoped)	Inline expansion (no implicit wrapp

8. Appendix: multi-CPU architecture

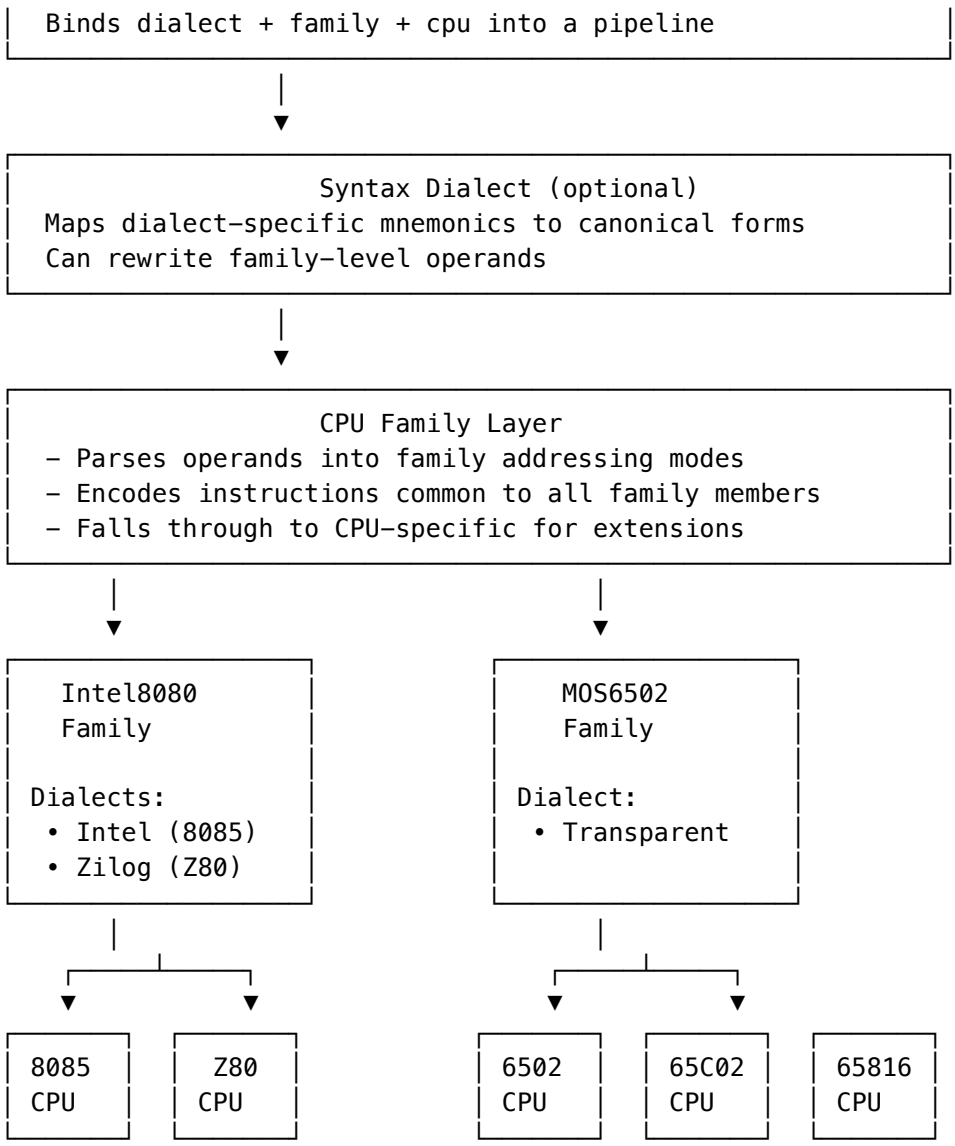
This appendix describes the modular architecture that allows opForge to support multiple CPU targets through a common framework, including Intel 8080-family CPUs, MOS 6502-family CPUs, Motorola 6800-family CPUs, and the shipped Motorola 68000-family lineage through `m68080`.

It is written for readers who want to understand how CPU support is organized internally, not just which directives are available at source level.

Overview

The assembler is organized into layers with hierarchical parsing and encoding. The diagram below is schematic and does not attempt to list every currently registered family or CPU:

```
┌──────────────────────────────────────────────────────┐
│                   Assembler Core                     │
│  (preprocessing, macro expansion, symbol table, output)  │
└──────────────────────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────────────────────┐
│                   Generic Parser                     │
│  Handles: labels, directives, macros, scopes, expressions  │
│  Extracts: mnemonic + operand expressions for instructions  │
│  Does NOT interpret addressing mode syntax           │
└──────────────────────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────────────────────┐
│                Module Registry                       │
│  Registers CPU families + CPU variants               │
```



Assembler pipeline (CPU family/dialect)

Read this pipeline from top to bottom: each stage narrows a parsed instruction from generic syntax into a family-specific form and finally into a concrete CPU encoding.

Registry

- Families and CPUs are registered in Rust at startup.
- CPU names are case-insensitive and resolved via the registry.

Selection

- `.cpu` <name> selects the active CPU.
- There is no **`.dialect`** directive.
- Dialect mapping (when present) is selected by CPU default or family canonical dialect.

Encoding pipeline

1. Parse operands with the family handler.
2. Apply dialect mapping (mnemonic + operand rewrite).
3. Attempt family pre-encode (optional).
4. Resolve operands with the CPU handler.
5. Run CPU validator (trait exists, currently unused by CPUs).
6. Encode with family handler; fall back to CPU handler for extensions.

VM hierarchy bridge (v1) See also:

- VM Boundary & Protocol Specification (v1) (host/VM boundary + strictness rules)
- VM Ultimate64 ABI Contract (v1) (constrained-runtime ABI for `*.opasm`)
- The VM runtime bridge provides host-facing target selection:
  - `set_active_cpu(cpu_id)`
  - `resolve_pipeline(cpu_id, dialect_override?)`
- Dialect override is host policy only; source still has no **`.dialect`** directive.
- The bridge validates override compatibility against family ownership + optional CPU allow-list.
- Dialect modules are rewrite-only surfaces; instruction encoding remains family/CPU owned.

VM runtime package source selection

- The CLI supports explicit VM runtime package loading with `--opasm-package <FILE>` (or `OPFORGE_OPASM_PACKAGE`).
- The high-level assembly/session path accepts the same kind of explicit `.opasm` package override through its request/config surface.
- Package source precedence for those paths is:
  1. explicit package path
  2. default artifact path `target/vm/opforge-vm-runtime.opasm` (when `vm-runtime-opasm-artifact` is enabled)
  3. bundled/runtime-generated fallback (disabled when `vm-runtime-opasm-unbundled` is enabled)
- In `vm-runtime-only` + `vm-runtime-opasm-unbundled` builds, runtime package sourcing is mandatory:
  - either provide `--opasm-package <FILE>`
  - or enable artifact mode and provide the default artifact file
- Lower-level engine/editor helpers such as `libopforge::processing`, `editor_parse_line`, and `editor_route_line` do not broaden this into CLI-style fallback behavior.
- In `vm-runtime-only` mode, those helpers should be called with an explicit runtime model via the `*_with_model` entrypoints, or with the default artifact file present when `vm-runtime-opasm-artifact` is enabled.
- If neither an explicit model nor the default artifact is available, those helpers report an unavailable runtime model instead of generating or bundling one implicitly.

VM boundary (certified families)

For certified families, the per-line assembly hot path uses VM contracts for tokenization, parser envelope execution, portable expression services, and authoritative encode tables.

Canonical host/VM boundary and failure/strictness rules:

- VM Boundary & Protocol Specification (v1)

Layer responsibilities

These layers are intentionally separated so that syntax, CPU selection, and byte emission can evolve without collapsing into one monolithic encoder.

Assembler Core

- File inclusion and preprocessing
- Output generation (listing files, hex files)
- Error collection and reporting
- Two-pass assembly coordination

Generic Parser

- Labels, directives, macros, scopes, expressions, comments
- Extracts instruction mnemonic + operand expressions
- Does not interpret addressing mode syntax

Module Registry

- Registers family and CPU handlers
- Resolves pipeline (family + CPU + dialect mapping)

Family Handler

- Parses family-common operand syntax
- Encodes family-common instructions
- Falls through to CPU handler for extensions

CPU Handler

- Resolves ambiguous operands
- Encodes CPU-specific instructions
- Validates CPU-specific constraints

Hierarchical processing

1. Generic parser extracts mnemonic + operand expressions.
2. Family handler parses expressions into family operands.
3. Dialect mapping rewrites mnemonic/operands (if needed).
4. Family pre-encode (optional) attempts encoding from family operands.
5. CPU handler resolves ambiguous operands to CPU-specific operands and applies CPU validation.
6. Encode with family handler first; CPU handler encodes extensions when family returns `NotFound`.

Family extensions

The tables below show the practical shape of family inheritance: a base family defines the common forms, and richer CPUs widen the accepted syntax or mnemonic set without changing the surrounding assembler model.

MOS 6502 Family

Operand syntax extensions:

Syntax	6502 (Base)	65C02 (Extended)
`#$20`	Immediate ✓	Immediate ✓
`$20`	Zero Page ✓	Zero Page ✓
`($20,X)`	Indexed Indirect ✓	Indexed Indirect ✓
`($20),Y`	Indirect Indexed ✓	Indirect Indexed ✓

Syntax	6502 (Base)	65C02 (Extended)
`($20)`	✗ Invalid	Zero Page Indirect ✓
`($1234,X)`	✗ Invalid	Absolute Indexed Indirect ✓

Instruction extensions:

Instruction	6502 (Base)	65C02 (Extended)
`LDA`	✓ All modes	✓ All modes + ($zp)
`BRA`	✗	✓ Branch Always
`PHX`, `PLX`	✗	✓ Push/Pull X
`PHY`, `PLY`	✗	✓ Push/Pull Y
`STP`, `WAI`	✗	✓ Stop/Wait
`DEC A`/`INC A`	✗	✓ Accumulator mode (`DEA`/`INA` aliases)
`BIT #imm`, `BIT zp,X`, `BIT abs,X`	✗	✓ Extended BIT modes
`STZ`	✗	✓ Store Zero
`TRB`, `TSB`	✗	✓ Test and Reset/Set Bits
`BBSn`, `BBRn`	✗	✓ Branch on Bit Set/Reset
`RMBn`, `SMBn`	✗	✓ Reset/Set Memory Bit

MOS 6502 Family (65816 additions)

Currently implemented 65816-specific additions:

- `BRL`, `JML`, `JSL`, `RTL`
- `JMP [$nnnn]` (alias for `JML [$nnnn]`)
- `REP`, `SEP`, `XCE`, `XBA`
- `PHB`, `PLB`, `PHD`, `PLD`, `PHK`, `TCD`, `TDC`, `TCS`, `TSC`
- `PEA`, `PEI`, `PER`, `COP`, `WDM`
- `MVN`, `MVP`
- operand forms: `d,S`, `(d,S),Y`, bracketed indirect (`[...]`, `[...,Y]`) for supported instructions
- runtime-state directives: see §3.11.2.1

Intel 8080 Family

Operand syntax extensions:

Syntax	8080/8085 (Base)	Z80 (Extended)
`A`, `B`, `HL`	Register ✓	Register ✓
`(HL)`	Indirect ✓	Indirect ✓
`(IX+d)`, `(IY+d)`	✗	CB-prefix targets only ✓ (general forms not yet encoded)

Instruction extensions:

The Z80 adds `DJNZ`, `JR` (with conditions), `EX`, `EXX`, block operations (`LDI`, `LDIR`, etc.), bit operations (`BIT`, `SET`, `RES`), and shifts/rotates including `SLL`. Z80-specific registers include `IX`, `IY`, `IXH`, `IXL`, `IYH`, and `IYL`.

Syntax dialects (8080 vs Z80)

This table is a reminder that opForge treats dialect differences as syntax surface over a shared underlying instruction family. The byte encoding is the same where the instruction semantics line up.

Operation	8080/8085 Dialect	Z80 Dialect	Opcode
Move register	`MOV A,B`	`LD A,B`	78
Move immediate	`MVI A,55h`	`LD A,55h`	3E 55
Load direct	`LDA 1234h`	`LD A,(1234h)`	3A 34 12
Store direct	`STA 1234h`	`LD (1234h),A`	32 34 12
Jump	`JMP 1000h`	`JP 1000h`	C3 00 10
Jump if zero	`JZ 1000h`	`JP Z,1000h`	CA 00 10
Call	`CALL 1000h`	`CALL 1000h`	CD 00 10
Return	`RET`	`RET`	C9
Add register	`ADD B`	`ADD A,B`	80
Add immediate	`ADI 10h`	`ADD A,10h`	C6 10

Core abstractions

- CpuType: concrete processor (I8085, Z80, M6502, M65C02, M65816, M45GS02, M6809, HD6309)
- CpuFamily: processor family (Intel8080, MOS6502, Motorola6800)

Handler traits (summary)

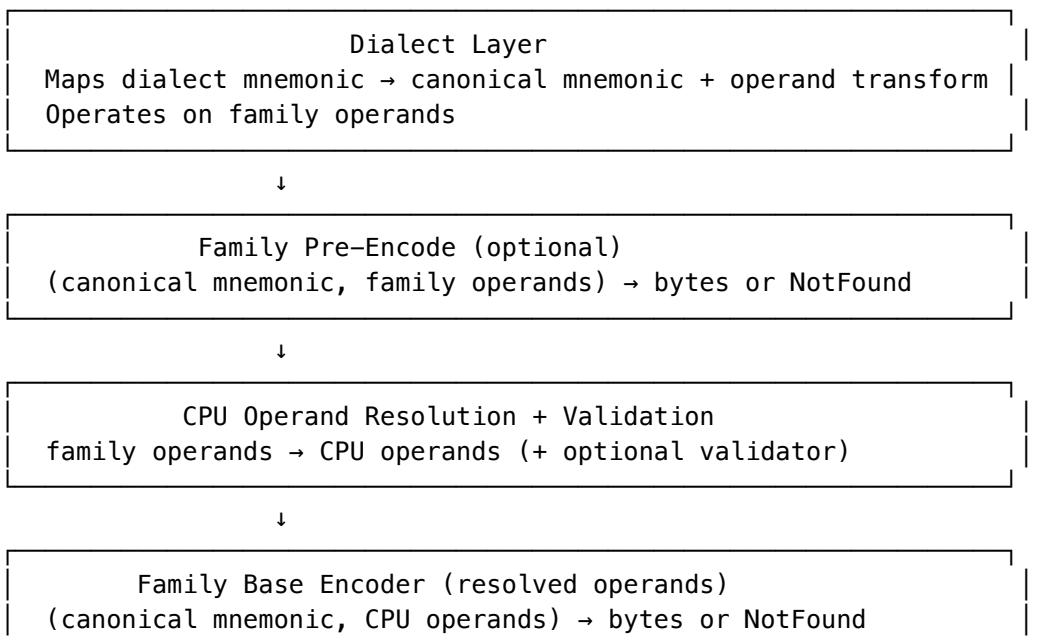
FamilyHandler provides:

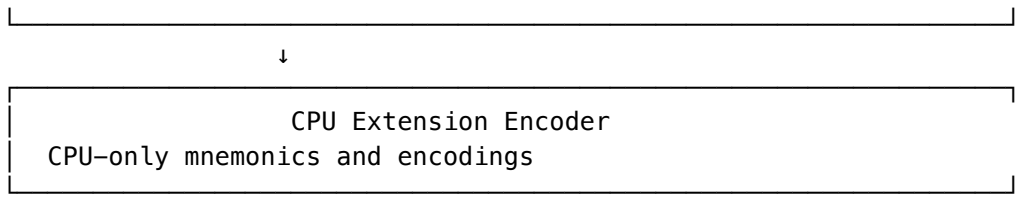
- Operand parsing for family-common syntax
- Optional pre-encoding using family operands
- Instruction encoding for family-common mnemonics
- Register and condition code recognition

CpuHandler provides:

- Resolution of ambiguous operands to CPU-specific forms
- Instruction encoding for CPU-specific mnemonics
- Query methods for supported mnemonics

Instruction resolution architecture





Module interfaces (summary)

```rust
pub trait FamilyModule: Send + Sync {
    fn family_id(&self) -> CpuFamily;
    fn family_cpu_id(&self) -> Option<CpuType> { None }
    fn family_cpu_name(&self) -> Option<&'static str> { None }
    fn cpu_names(&self, registry: &ModuleRegistry) -> Vec<String>;
    fn canonical_dialect(&self) -> &'static str;
    fn dialects(&self) -> Vec<Box<dyn DialectModule>>;
    fn handler(&self) -> Box<dyn FamilyHandlerDyn>;
}

pub trait CpuModule: Send + Sync {
    fn cpu_id(&self) -> CpuType;
    fn family_id(&self) -> CpuFamily;
    fn cpu_name(&self) -> &'static str;
    fn default_dialect(&self) -> &'static str;
    fn handler(&self) -> Box<dyn CpuHandlerDyn>;
    fn validator(&self) -> Option<Box<dyn CpuValidator>> { None }
}

pub trait DialectModule: Send + Sync {
    fn dialect_id(&self) -> &'static str;
    fn family_id(&self) -> CpuFamily;
    fn map_mnemonic(
        &self,
        mnemonic: &str,
        operands: &dyn FamilyOperandSet,
    ) -> Option<(String, Box<dyn FamilyOperandSet>)>;
}
```

9. Appendix: AmigaOS Hunk symbolic notation boundary

This appendix gives the short reader-facing rule for the current executable Hunk notation boundary.

For `format=hunk`, opForge accepts natural bare-symbol instruction spelling only for the frozen one-symbol subset where:

- there is exactly one relocatable symbol-bearing operand
- the assembler has one canonical absolute-long encoding for that form
- that encoding maps directly onto the current executable `HUNK_RELOC32` support

Representative covered forms include:

```
.long target
LEA target,A1
MOVE.L D0,target
ADDI.W #1,target
```

```
MOVEM.L target,D1/D3
CAS.W D0,D1,target
BFTST target{3:5}
```

Representative explicit-only or unsupported forms include:

```
MOVE.L target1,target2
MOVE.L #target+4,D1
MOVE.L (target,A0),D0
```

In practice, if a symbolic instruction form is not part of the frozen one-symbol executable subset, opForge does not guess. It fails explicitly with either an "explicit `.L` notation required" diagnostic or an "unsupported symbolic Hunk instruction form in v`0.3`" diagnostic, depending on whether the shape is ambiguous or outside the supported relocation model.

The authoritative full compatibility matrix lives in opForge-amiga-hunk-executable-completeness-spec-v0_3.md.

10. Appendix: VM package authoring notes (v0.1 draft)

See also:

- VM Boundary & Protocol Specification (v1)
- VM Ultimate64 ABI Contract (v1)

Ownership rules

- Put canonical/shared instruction forms in family scope.
- Put CPU-only mnemonics, overrides, and capability-specific forms in CPU scope.
- Put syntax rewrites only in dialect scope (mnemonic/operand/token rewrites).
- Dialects never encode bytes directly; they rewrite into canonical family/CPU encode paths.

Compatibility and fallback rules

- Dialect namespace is family-owned.
- `resolve_pipeline` order is: explicit host override -> CPU default dialect -> family canonical dialect.
- Optional dialect CPU allow-lists are enforced after selection.
- Source files have no `.dialect` directive in compatibility mode.

Native-to-package migration checklist

1. Extract shared forms/registers from family handlers into family-scoped package metadata.
2. Move CPU extension forms/registers into CPU-scoped overlays.
3. Convert dialect mapper tables into deterministic rewrite rules.
4. Keep directives/macros/linker/output behavior host-owned in v0.1.
5. Add parity vectors (`.optst`) and run `make test-vm-parity`.